PROJECT ATLAS — AEROSPACE GROUND SYSTEMS

SDS-001: Software Design Specification

Master Flightline Clock System — Version 2.0

Document Ref:	ATLAS-SDS-001
Version:	2.0.0
Status:	Released
Date:	3 July 2026
Author:	Atlas Development Team
Classification:	Internal / Restricted

Table of Contents

1. Document Control 3

1.1 Revision History 3

2. Executive Summary 4

3. System Architecture 5

3.1 Architectural Overview 5

3.2 Package Structure 5

3.3 Design Patterns 6

4. Software Components 7

4.1 Core Package 7

4.2 GUI Package 8

4.3 HAL Components 9

5. Hardware Abstraction Layer (HAL) Design 9

5.1 Backend Interface Specification 9

5.2 Backend Selection Mechanism 10

5.3 Extensibility 11

6. User Interface Design 12

6.1 Settings Tab Enhancements 12

6.2 Platform-Aware Elements 13

7. Configuration Management 13

8. Startup and Boot Sequence 14

9. Data Flow and State Management 16

10. Cross-Platform Compatibility 17

11. Testing Strategy 18

12. Future Enhancements 20

1. Document Control

Document Reference:	ATLAS-SDS-001
Document Title:	Software Design Specification — Master Flightline Clock
Version:	2.0.0
Status:	Released
Date of Issue:	3 July 2026
Project:	Project Atlas — Master Flightline Clock
Author:	Atlas Development Team
Owner:	Peter Lochhead Hepburn
Confidentiality:	Internal / Restricted — Not for external distribution
Related Documents:	ATLAS-ICD-001 (Interface Control Document), ATLAS-URS-001 (User Requirements), ATLAS-TM-001 (Traceability Matrix)

1.1 Revision History

Table 1 Revision History

Version	Date	Author	Description of Changes
1.0.0	2026-06-15	Atlas Dev Team	Initial release. Core engine, SQLite persistence, LED simulator, and basic operator console documented.
1.1.0	2026-06-22	Atlas Dev Team	Added 6-Row Adaptive Display rendering, compact mode for 64×64 panels, brightness slider with environmental labels.
1.5.0	2026-07-02	Atlas Dev Team	Introduced custom navigation bar replacing native QTabWidget. Added graceful restart and shutdown controls. Documented stable v1.5.0 appliance baseline.

2.0.0	2026-07-03	Atlas Dev Team	Major architectural revision. Introduced Hardware Abstraction Layer (HAL) with `DisplayBackend` abstract interface. Added `SimulatorBackend` and `HardwareBackend` stub. Settings tab enhanced with Display Type dropdown and Auto-Boot toggle. Platform-aware Exit button. Auto-boot to clock mode on system start. New persisted settings: `display_backend_type`, `auto_boot_enabled`. Code reorganised into `/core` and `/gui` packages. Cross-platform compatibility verified on macOS, Windows, and Linux.

2. Executive Summary

Project Atlas is a dedicated competition appliance providing real-time flightline timing and display management for competition aviation environments. This document — ATLAS-SDS-001 version 2.0 — supersedes all prior versions and describes the complete software architecture as implemented following the July 2026 architectural revision.

The defining characteristic of v2.0 is the transition from a conventional desktop application to a purpose-built **appliance-mode operating model**. On system power-up, Atlas initialises autonomously, enters clock mode without user intervention, and begins monitoring the heat schedule. The operator console remains accessible for schedule management and configuration, but the system's natural state is active and display-ready — not idle at a menu.

This release introduces the **Hardware Abstraction Layer (HAL)**, a formal software interface that decouples the timing engine and operator console from any specific physical output medium. The HAL defines a `DisplayBackend` abstract class with a uniform update API; the runtime selects between a `SimulatorBackend` (on-screen Qt renderer) and a `HardwareBackend` stub (future physical LED panels) via a factory function driven by the persisted `display_backend_type` setting. This design ensures that the transition from software simulation to physical LED hardware will require changes only to the backend implementation — not to the engine, operator console, or data model.

The software stack is implemented in Python using the PySide6 (Qt6) framework. SQLite provides all persistent state storage, eliminating manual JSON configuration files for operational parameters. Cross-platform compatibility has been verified on macOS (primary deployment target: Mac Mini M4), Windows 10/11, and Ubuntu Linux.

> **Scope of this document:** ATLAS-SDS-001 v2.0 covers the complete software architecture, component design, HAL specification, UI layout, configuration schema, boot sequence, data-flow model, cross-platform strategy, test approach, and future roadmap for the Atlas v2.0 release. Interface signal-level specifications for the physical 6-row LED panel are addressed separately in ATLAS-ICD-001.

3. System Architecture

3.1 Architectural Overview

The Atlas v2.0 architecture is structured as a layered system. Each layer has a single responsibility and communicates with adjacent layers through defined interfaces. No layer reaches across a boundary — the engine does not manipulate display widgets, and display backends do not read from the database directly.

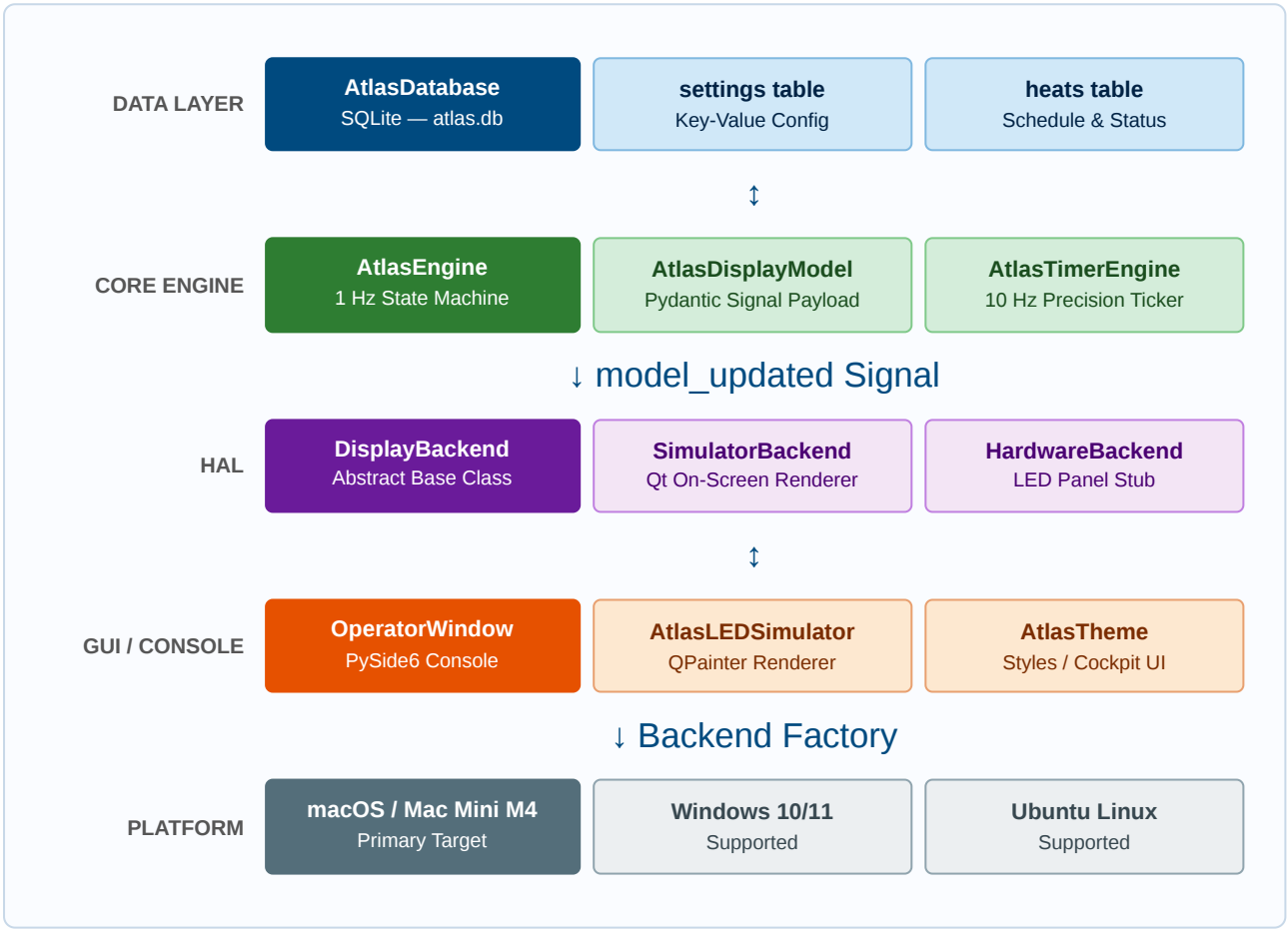


Figure 1 Atlas v2.0 Layered Architecture — HAL decouples engine from display medium

3.2 Package Structure

The codebase is organised into two Python packages beneath a `src/` root, with `main.py` serving as the application entry point and wiring harness. This separation enforces clean import boundaries: the `core` package has no GUI imports; the `gui` package imports from `core` but not vice versa.

```
Project_Atlas/
├── main.py                    # Entry point — wires all subsystems
├── run_atlas.sh               # macOS/Linux launch script
├── data/
│   └── atlas.db               # SQLite database (runtime-generated)
├── core/
│   ├── __init__.py
│   ├── models.py              # Pydantic data models (HeatConfig,
AtlasDisplayModel)
│   ├── database.py            # AtlasDatabase — SQLite CRUD and settings
persistence
│   ├── engine.py              # AtlasEngine — 1 Hz state machine, Signal
broadcaster
│   ├── timers.py              # AtlasTimerEngine — 10 Hz precision countdown
│   └── config.py              # AtlasConfig — JSON fallback / hardware baseline
└── gui/
    ├── __init__.py
    ├── windows.py             # OperatorWindow — tabbed operator console (PySide6)
    ├── led_simulator.py       # AtlasLEDSimulator — QPainter LED matrix renderer
    ├── display_backend.py     # DisplayBackend ABC, SimulatorBackend,
HardwareBackend
    ├── themes.py              # AtlasTheme — cockpit stylesheet constants
    └── fonts.py               # AtlasBitmapFont — 5×7 pixel font engine
```

3.3 Design Patterns

Atlas v2.0 employs three interlocking design patterns that together allow each subsystem to evolve independently:

Table 2 Design Patterns Applied in Atlas v2.0

Pattern	Where Applied	Rationale
Model-View-Controller (MVC)	Engine (model) → HAL/GUI (view) → Operator console (controller)	Business logic (engine) is fully isolated from presentation; swapping view layers requires no engine changes.
Factory / Strategy	`display_backend.py` — `create_backend()` function	Runtime selects concrete backend (`SimulatorBackend` or `HardwareBackend`) from persisted configuration without conditional logic scattered through the application.
Observer (Qt Signals)	`AtlasEngine.model_updated` signal	Engine broadcasts `AtlasDisplayModel` to any subscriber — simulator, hardware backend, and operator console all receive the same update without the engine holding references to any of them.

4. Software Components

4.1 Core Package (`core/`)

Table 3 Core Package Component Summary

Module	Class / Object	Responsibility
`engine.py`	`AtlasEngine`	Central 1 Hz state machine. Reads the active heat from the database each tick, computes phase and countdown, populates an `AtlasDisplayModel`, and broadcasts it via the `model_updated` Qt signal. Also exposes `cancel_active_heat()` for operator abort and `brightness_changed` signal for live dimming without restart.
`database.py`	`AtlasDatabase`	SQLite abstraction layer. Bootstraps schema on first run (`heats` and `settings` tables). Provides typed CRUD methods: `save_heat()`, `get_active_heat()`, `get_next_pending_heat()`, `set_heat_status()`, `cancel_active_heat()`, `save_setting()`, `get_setting()`. All operations use parameterised queries; no raw string interpolation of user-supplied data.
`models.py`	`HeatConfig`, `AtlasDisplayModel`	Pydantic v2 data models. `HeatConfig` carries all schedule parameters for one heat. `AtlasDisplayModel` is the immutable signal payload emitted each engine tick; it fully describes what any backend should render — no backend needs to query the database independently.
`timers.py`	`AtlasTimerEngine`	10 Hz precision ticker using a `QTimer` at 100 ms intervals. Computes real-time remaining seconds against a UTC `target_end_time`, emitting a `tick(str)` signal (MM:SS format) and a `finished()` signal on expiry. Decoupled from the main engine to allow sub-second precision for display animation use-cases.

`config.py`	`AtlasConfig`	JSON-based hardware baseline configuration. Stores physical panel geometry defaults (`panels_wide` , `panels_high` , `led_scale`) for the initial boot before SQLite settings are populated. Computes derived properties `display_width` and `display_height` from panel counts. This module is a legacy compatibility shim; runtime configuration is progressively migrated to the SQLite settings table.

4.2 GUI Package (`gui/`)

Table 4 GUI Package Component Summary

Module	Class / Object	Responsibility
`windows.py`	`OperatorWindow`	Primary operator-facing Qt `QMainWindow` . Implements three-tab navigation (Live, Schedule, Settings) via a custom `QStackedWidget` and styled push-button navigation bar, bypassing native macOS tab-bar styling constraints. Subscribes to `AtlasEngine.model_updated` for live status display. Implements restart (`os.execv`) and graceful shutdown (`QApplication.quit()`) controls.
`led_simulator.py`	`AtlasLEDSimulator`	Software LED panel renderer. Uses Qt `QPainter` to draw each LED as an ellipse at a configurable scale factor. Implements Full layout (128×64) and Compact layout (64×64) automatically based on display width. Renders six rows: local time, heat info, open time, close time, phase label, primary countdown. Supports live brightness scaling via `set_brightness()` . Subscribes to `model_updated` signal.
`display_backend.py`	`DisplayBackend` , `SimulatorBackend` , `HardwareBackend`	HAL implementation module. Defines the `DisplayBackend` ABC and two concrete implementations. See Section 5 for full interface specification.
`themes.py`	`AtlasTheme`	Centralised stylesheet constants for the cockpit UI aesthetic. Provides `COCKPIT_BG` , `BTN_MASTER_ARM` , and `BTN_CANCEL` QSS strings. Tab stylesheet (`TABS`) retained as legacy reference; active navigation uses the custom button bar.

`fonts.py`	`AtlasBitmapFont`	Custom 5×7 pixel bitmap font engine. Stores a glyph map (`FONT_5x7`) as column-encoded integer arrays. The LED simulator uses this font exclusively to ensure pixel-perfect fidelity matching physical P-series LED panel font rendering. Character set covers A–Z, 0–9, and common punctuation.

4.3 HAL Components

The Hardware Abstraction Layer is implemented in `gui/display_backend.py` and comprises three classes: the abstract base, the simulator implementation, and the hardware stub. A factory function `create_backend(backend_type, config, engine)` instantiates the appropriate concrete class at startup based on the `display_backend_type` setting.

Table 5 HAL Class Summary

Class	Type	Description
`DisplayBackend`	Abstract Base Class	Defines the contract all backends must honour. Specifies `update(model: AtlasDisplayModel)` , `initialise()` , `shutdown()` , and `set_brightness(value: int)` as abstract methods.
`SimulatorBackend`	Concrete — Production	Wraps `AtlasLEDSimulator` . Forwards `update()` calls to `led_sim.update_model()` and `set_brightness()` to `led_sim.set_brightness()` . Used when `display_backend_type = "simulator"` .
`HardwareBackend`	Concrete — Stub	Placeholder for physical LED panel communication. `initialise()` opens a serial/network connection (not yet implemented). `update()` will serialise the model to the ICD-001 frame format and transmit. Returns without error in current stub form — graceful fallback assured.

5. Hardware Abstraction Layer (HAL) Design

5.1 Backend Interface Specification

The `DisplayBackend` abstract base class defines the complete contract that all display backends must implement. The interface is intentionally minimal and model-driven: the engine provides a fully-populated `AtlasDisplayModel` ; the backend decides how to render it. No backend receives raw timer values or database handles.

```
from abc import ABC, abstractmethod
from core.models import AtlasDisplayModel

class DisplayBackend(ABC):
    """Abstract base class for all Atlas display backends.
    All concrete backends must implement every method below.
    """

    @abstractmethod
    def initialise(self) -> None:
        """Open hardware connection or create Qt widget.
        Called once at application startup, before first update().
        Must not raise — log and continue on connection failure."""
        ...

    @abstractmethod
    def update(self, model: AtlasDisplayModel) -> None:
        """Render the current display model.
        Called once per engine tick (nominally 1 Hz).
        Must complete within 200 ms to avoid blocking the Qt event loop."""
        ...

    @abstractmethod
    def set_brightness(self, value: int) -> None:
        """Set display brightness. value is in range 10–100 (percent).
        Called on live slider change — must not require restart."""
        ...

    @abstractmethod
    def shutdown(self) -> None:
        """Gracefully close hardware link or destroy Qt widget.
        Called during application closeEvent. Must transmit clear-screen
        frame to physical panels before disconnecting."""
        ...
```

5.2 Backend Selection Mechanism

The backend is selected at startup via a factory function. The selection is driven by the `display_backend_type` key in the SQLite `settings` table. If the key is absent or its value is unrecognised, the factory defaults to `SimulatorBackend` to ensure the system always renders something visible — a critical safety property for appliance-mode operation where no user may be present to diagnose a blank display.

```
def create_backend(backend_type: str, config, engine) -> DisplayBackend:
    """Factory function — instantiates and returns the correct backend.

    Args:
        backend_type: "simulator" | "hardware" (from settings table)
        config:       DisplayConfig object (panel dimensions, scale)
        engine:       AtlasEngine (required by SimulatorBackend for signal wiring)

    Returns:
        Initialised DisplayBackend instance.
    """
    if backend_type == "hardware":
        backend = HardwareBackend(config)
    else:
        backend = SimulatorBackend(config, engine)   # default / fallback

    backend.initialise()
    return backend
```

The operator selects the backend via the **Display Type** dropdown in the Settings tab (Section 6.1). The selection persists to `display_backend_type` in SQLite. A restart is required to activate the new backend; the UI prompts accordingly.

5.3 Extensibility

The factory pattern ensures that adding a new backend type (e.g., `ArtnetBackend` for DMX/Artnet LED controllers, or `SerialBackend` for RS-485 P10 panel strings) requires only:

1. Implementing the four abstract methods of `DisplayBackend` in a new class.
2. Adding the new class identifier to the `create_backend()` factory.
3. Adding the new option to the Display Type dropdown in `windows.py`.

No changes to the engine, data models, database schema, or operator console logic are required. The HAL boundary is the sole modification surface for new output media.

> **Interface Control Document:** The serialisation format for physical LED panel frames — row ordering, colour encoding, brightness byte, and framing — is defined in ATLAS-ICD-001. The `HardwareBackend.update()` implementation will consume `AtlasDisplayModel` fields and encode them per ICD-001 before transmission.

6. User Interface Design

The Atlas operator console is implemented as a full-screen-capable `QMainWindow` (default size 900×850 px) with a custom navigation bar replacing native OS tab controls. The cockpit aesthetic — high-contrast black background, cyan and blue accents — is deliberately designed for readability under direct sunlight at an outdoor competition site.

6.1 Settings Tab Enhancements

The Settings tab was substantially extended in v2.0 to expose the new HAL and appliance-mode controls. The following table documents every control on the Settings tab, its type, bound setting key, and behaviour.

Table 6 Settings Tab Controls (v2.0)

Control Label	Widget Type	Setting Key	Behaviour
Competition Name	`QLineEdit`	`competition_name`	Free-text label displayed on display header rows. Persisted on Save.
Display Size	`QComboBox`	`display_width` / `display_height`	Selects 128×64 (Full layout) or 64×64 (Compact layout). Applies after restart.
Display Type	`QComboBox`	`display_backend_type`	Selects backend: "LCD Monitor (Simulator)" maps to `simulator`; "LED Module (Hardware)" maps to `hardware`. Applies after restart via factory.
Auto-Boot	`QCheckBox` / Toggle	`auto_boot_enabled`	When ON, `main.py` skips the operator console at launch and goes directly to clock/display mode. Toggle takes effect on next start.
Sim LED Scale	`QLineEdit`	`led_scale`	Integer pixel scale factor for the software simulator (e.g., 8 = each LED dot is 8×8 px). Applies after restart.
Display Intensity	`QSlider` (horizontal)	`brightness`	Range 10–100%. Live update: slider valueChanged signal immediately calls `engine.brightness_changed.emit(value)`, which backends handle without restart. Environmental label shown: Night / Overcast / Normal / Bright Day / Full Sun.

SAVE & APPLY	`QPushButton`	—	Writes all field values to the SQLite settings table. Shows confirmation dialog. Hardware-dependent settings note restart requirement.
RESTART ATLAS	`QPushButton`	—	Calls `os.execv(sys.executable, ['python3'] + sys.argv)` after user confirmation — in-process restart preserving all saved settings.
EXIT TO [PLATFORM]	`QPushButton`	—	Platform-aware label (see §6.2). Calls `QApplication.quit()` after user confirmation. Triggers `closeEvent` for graceful hardware disconnect.

6.2 Platform-Aware UI Elements

To reinforce the appliance-mode identity of the system, certain UI elements adapt their labelling to the host platform at runtime. The exit button is the primary example: on macOS it reads **"EXIT TO macOS"**, on Windows **"EXIT TO DESKTOP"**, and on Linux **"EXIT TO DESKTOP"**. This is achieved via `sys.platform` detection at `OperatorWindow` construction time.

The Fusion Qt style (`QStyleFactory.create("Fusion")`) is applied unconditionally in `main.py` to ensure that all widget rendering — sliders, combo boxes, check boxes — appears identically on macOS, Windows, and Linux. This eliminates platform-specific tab-bar rendering issues that affected earlier versions using the native macOS style with `QTabWidget`.

Platform detection is also used to resolve the default database path (`data/atlas.db`) relative to the application working directory, ensuring that the path is valid on all three operating systems without requiring separate configuration.

7. Configuration Management

All runtime configuration in Atlas v2.0 is stored in the SQLite `settings` table as key-value pairs. This approach provides atomic writes, crash-safe persistence, and a single source of truth for all operational parameters. Manual JSON editing for operational settings has been eliminated.

7.1 Settings Schema

Table 7 Settings Table — Key-Value Schema (v2.0)

Key	Type	Default	Description
`competition_name`	string	*empty*	Event name displayed on LED panel header.
`display_width`	integer	128	Active display width in LED pixels. Drives Full/Compact mode selection.

`display_height`	integer	64	Active display height in LED pixels.
`led_scale`	integer	8	Pixel scale multiplier for the software simulator window.
`brightness`	integer	80	Display brightness percentage (10–100). Applied live via signal.
`display_backend_type`	string	`simulator`	**New v2.0.** Backend identifier. Valid values: `simulator`, `hardware`. Factory defaults to `simulator` if absent or invalid.
`auto_boot_enabled`	string (bool)	`False`	**New v2.0.** When `True`, `main.py` enters clock/display mode at startup without showing the operator console. Stored as string "True"/"False" for SQLite compatibility.

7.2 Persistence Strategy

The `AtlasDatabase.save_setting(key, value)` method uses an SQLite `INSERT OR REPLACE` statement, which is atomic at the SQLite journal level. A crash mid-write will leave either the old value or the new value — never a corrupt intermediate. Settings are read at startup via `get_setting(key, default)`, which returns the default if the key is absent, making the schema forward-compatible: new settings introduced in future versions will always have a valid default on first run.

7.3 Cross-Platform Path Handling

The database path (`data/atlas.db`) is resolved relative to the current working directory at process start. The `AtlasDatabase.__init__()` method calls `os.makedirs("data", exist_ok=True)` before opening the connection, ensuring the directory is created on first run regardless of platform. The `AtlasConfig` module (JSON baseline) similarly resolves `atlas_config.json` relative to the working directory via `os.path.exists()`. All path operations use `os.path` rather than hard-coded separators.

8. Startup and Boot Sequence

The v2.0 boot sequence has been redesigned to support appliance-mode operation. The system initialises all subsystems before displaying any window, so the first user-visible state is always a functioning display — never a loading screen or splash page.

8.1 Normal Boot Flow

```
main.py start
    │
    ├─ 1. QApplication initialised (Fusion style forced)
    │
    ├─ 2. AtlasDatabase.bootstrap()
    │       Connects to data/atlas.db
    │       Creates heats + settings tables if absent
    │
    ├─ 3. AtlasEngine(db) constructed
    │       QTimer started at 1000 ms interval
    │       First tick fires immediately on event loop start
    │
    ├─ 4. DisplayConfig loaded from settings table
    │       (display_width, display_height, led_scale, brightness)
    │
    ├─ 5. auto_boot_enabled check
    │       ├─ True  → skip OperatorWindow, proceed directly to step 6
    │       └─ False → OperatorWindow.show() [operator console visible]
    │
    ├─ 6. create_backend(display_backend_type, config, engine)
    │       ├─ "simulator" → SimulatorBackend.initialise() → led_sim.show()
    │       └─ "hardware"  → HardwareBackend.initialise() → connect to panels
    │                          (fallback to SimulatorBackend on connection failure)
    │
    ├─ 7. Signal wiring
    │       engine.model_updated  → backend.update()
    │       engine.brightness_changed → backend.set_brightness()
    │
    └─ 8. app.exec() — Qt event loop running
            Engine ticks at 1 Hz, emitting model_updated
```

8.2 Graceful Fallback on Hardware Failure

If `HardwareBackend.initialise()` raises or fails to establish a panel connection, the factory catches the exception, logs a warning, and constructs a `SimulatorBackend` in its place. This ensures the system never boots to a blank state — the software simulator serves as the operational fallback, allowing the competition to continue while hardware is investigated. The operator console Settings tab will show an amber indicator that the hardware backend failed to initialise.

8.3 Shutdown Sequence

When the operator initiates shutdown (EXIT button or window close), `OperatorWindow.closeEvent()` fires. The backend's `shutdown()` method is called, which — in the hardware case — transmits a clear-screen frame to the LED panels before disconnecting,

ensuring physical panels do not display stale timing data after the software exits. `QApplication.quit()` then terminates all windows cleanly.

9. Data Flow and State Management

9.1 Engine Tick Cycle

The primary data flow is a unidirectional broadcast loop driven by the `AtlasEngine` at 1 Hz. The engine does not hold references to any display component; it publishes a value-typed `AtlasDisplayModel` and any subscriber may consume it. This ensures the engine remains testable in isolation — no GUI or hardware dependencies are required to unit-test state transitions.

```
AtlasDatabase (SQLite) │ get_active_heat() / get_next_pending_heat() ▼
AtlasEngine._update_state() │ Computes phase, countdown, colours → populates
AtlasDisplayModel ▼ AtlasEngine.model_updated.emit(model) ← Qt Signal broadcast ├──►
OperatorWindow.refresh(model) ← Updates heat_status label ├──►
SimulatorBackend.update(model) ← Triggers LED QPainter repaint └──►
HardwareBackend.update(model) ← Encodes ICD-001 frame, transmits
```

9.2 AtlasDisplayModel — Signal Payload

The `AtlasDisplayModel` is a Pydantic `BaseModel` instance used as the signal payload. It is value-typed (immutable in intent) and carries all information needed to render any display variant without further database access.

Table 8 AtlasDisplayModel Fields

Field	Type	Default	Description
`local_time`	str	"00:00:00"	Current local time, HH:MM:SS format.
`primary_timer`	str	"--:--"	Active countdown in MM:SS format, or "--:--" when idle.
`primary_timer_label`	str	"SYSTEM IDLE"	Phase label: TRACK OPENS IN / ENTER NOW / HEAT TIME REMAINING / WAITING.
`primary_timer_color`	str	"#8888"	Hex colour for timer display — yellow (pre-entry), green (entry), red (active heat).
`band_color`	str	"#3333"	Background accent colour for the bottom status band row.
`heat_number`	str	"--"	Current heat identifier string (e.g., "01", "02A").

`thermalling_dir`	str	"---"	Thermalling direction for current heat: LEFT or RIGHT.
`is_armed`	bool	False	True when an active or imminent heat is in scope. Drives arm/idle display branching.
`planned_open`	str	"--:--"	Scheduled track open time, HH:MM format, for static display row.
`planned_close`	str	"--:--"	Scheduled track close time, HH:MM format, for static display row.

9.3 Settings Persistence Flow

Settings writes are triggered exclusively by the operator via the SAVE & APPLY button. The flow is: operator edits widget → presses SAVE → `OperatorWindow._save_settings()` calls `db.save_setting(key, value)` for each changed field → SQLite `INSERT OR REPLACE` executes atomically → confirmation dialog shown. On next restart, `main.py` reads the new values from the database and applies them to the `DisplayConfig` object and the backend factory.

10. Cross-Platform Compatibility

Atlas v2.0 is verified to run on macOS, Windows 10/11, and Ubuntu Linux. The primary deployment target is a dedicated Mac Mini M4 running macOS; the other platforms are supported for development and testing.

Table 9 Platform Compatibility Matrix

Feature	macOS (M4)	Windows 10/11	Ubuntu Linux
Qt Fusion style enforcement	Required — eliminates native tab rendering	Optional but applied for consistency	Optional but applied
Database path (`data/atlas.db`)	Relative to CWD, auto-created	Same — backslash handled by `os.path`	Same
Process restart (`os.execv`)	Supported — native POSIX exec	Supported — Python subprocess layer	Supported
Exit button label	"EXIT TO macOS"	"EXIT TO DESKTOP"	"EXIT TO DESKTOP"
HardwareBackend (serial/network)	Primary target for HW integration	USB-serial supported	USB-serial supported
SimulatorBackend	Full — all display sizes	Full	Full

Bitmap font rendering	QPainter — platform-independent	Same	Same

10.1 Platform Detection

Platform identity is determined at runtime using `sys.platform`:

- `darwin` → macOS. Fusion style enforced; Exit button reads "EXIT TO macOS".
- `win32` → Windows. Exit button reads "EXIT TO DESKTOP".
- `linux` → Linux. Exit button reads "EXIT TO DESKTOP".

10.2 Dependency Stack

Table 10 Runtime Dependency Stack

Dependency	Version	Purpose
Python	3.11+	Runtime. f-strings, `os.execv`, `sys.platform`.
PySide6 (Qt6)	6.6+	GUI framework. QApplication, QMainWindow, QPainter, QTimer, Signals.
Pydantic	v2.x	Data model validation. `AtlasDisplayModel`, `HeatConfig`.
SQLite3	stdlib	Persistence. Included with Python standard library — no install required.

11. Testing Strategy

Atlas v2.0 adopts a pragmatic testing strategy appropriate to a small-team, single-operator appliance context. The emphasis is on integration-level tests that validate full data paths — from database to display model to backend — rather than exhaustive unit tests of individual helper functions.

11.1 Test Categories

Table 11 Test Coverage Plan

Category	Coverage Target	Tool / Approach
Engine State Machine	All five operational phases (idle, waiting, pre-entry, entry, active heat, heat end)	pytest — instantiate `AtlasEngine` with an in-memory SQLite database (`:memory:`); seed heats at known UTC offsets; assert `AtlasDisplayModel` fields after manual `tick()` calls.

Database CRUD	All public methods of `AtlasDatabase`	pytest with a temporary file database. Assert read-back equality for `save_heat()` / `get_active_heat()`; verify `INSERT OR REPLACE` idempotency for settings.
Backend Switching	Factory produces correct class; fallback to simulator on hardware failure	pytest with monkeypatched `HardwareBackend.initialise()` raising `ConnectionError`; assert factory returns `SimulatorBackend`.
Offscreen Qt Rendering	Simulator renders without exceptions; brightness affects paint output	pytest-qt with `QT_QPA_PLATFORM=offscreen` environment variable. Assert `AtlasLEDSimulator` constructs and `update_model()` does not raise. Pixel sampling not required.
Settings Persistence	All new v2.0 keys round-trip correctly	pytest — write then read each new setting key (`display_backend_type`, `auto_boot_enabled`) and assert value equality.
Platform Detection	Exit button label adapts to platform	pytest with monkeypatched `sys.platform`; assert `OperatorWindow` exit button text.

11.2 Test Execution Environment

Tests are executed via `pytest` from the project root with `QT_QPA_PLATFORM=offscreen` to suppress display requirements on CI/CD runners. All Qt-dependent tests use the `pytest-qt` plugin to manage the `QApplication` lifecycle. Hardware backend tests use monkeypatching and do not require physical LED hardware.

11.3 Acceptance Criteria

The following integration-level acceptance criteria must pass before each release:

1. Engine transitions from IDLE → WAITING → ACTIVE on a seeded heat schedule without manual intervention.
2. Active heat auto-cancels (status set to COMPLETED) when `heat_end` timestamp passes.
3. Display backend switches correctly from simulator to hardware mode and back after restart.
4. Auto-boot flag causes application to skip operator console on next launch.
5. Brightness slider change reflects immediately in simulator without requiring a restart.
6. Application restarts cleanly via the Restart button with all settings preserved.

12. Future Enhancements

This section records planned enhancements and the roadmap for completing the transition from appliance prototype to certified flightline equipment. Items are listed in approximate priority order for the next development phase.

12.1 Physical LED Module Integration

The highest-priority next item is completing `HardwareBackend` to drive physical P-series LED panels. This will involve:

- Selecting and testing a communication protocol: serial (RS-485, UART) or network (UDP broadcast, Artnet DMX512-A).
- Implementing the ICD-001 frame serialiser inside `HardwareBackend.update()`.
- Validating timing latency — the 200 ms budget defined in the backend interface specification must be met under loaded panel conditions.
- Implementing `shutdown()` clear-screen frame to ensure panels go dark on application exit.

12.2 Additional Backend Types

Table 12 Planned Backend Extensions

Backend	Protocol	Use Case
`ArtnetBackend`	Artnet / DMX512-A (UDP)	Wireless LED wall panels at larger venues. No cable runs required.
`SerialBackend`	RS-485 / UART	Wired P10 LED panel strings. Highest reliability; suitable for primary flightline unit.
`NetworkDisplayBackend`	TCP/WebSocket	Remote display on a second screen (e.g., judges' table, pits area). Broadcasts display model JSON over LAN.
`OBSOverlayBackend`	WebSocket (OBS Studio)	Overlay timing data on live video stream for online broadcast of competitions.

12.3 Operational Enhancements

- **Automatic font scaling and centring:** Dynamically compute character width to centre strings on any panel width without hardcoded pixel offsets.
- **Competition management:** Multi-round scheduling, day-of-week heat templates, and carry-forward results between rounds.

- **Diagnostics page:** Live signal metrics, last engine tick timestamp, backend health indicator, and database row counts.
- **Configuration backup / import / export:** Export SQLite settings and heat schedule to a JSON archive; restore from backup on re-deployment.
- **Installer:** macOS .dmg package or automated setup script for headless Mac Mini M4 deployment with auto-login and launch-at-boot via LaunchAgent.
- **GAMP 5 Traceability:** Finalise ATLAS-TM-001 traceability matrix linking each requirement in URS-001 to a design element in this SDS and a test case in the verification plan.